

WillLead MVP 实现文档

目标是在 7 到 10 天内做出一个稳定可演示的 reactive-native autonomous wallet 原型，证明“用户只设置一次意图，之后即使前端离线，钱包仍然可以基于链上事件继续执行”。

1. 项目目标

WillLead 的 MVP 不是做一个全功能钱包，而是做一个最小可验证的新钱包范式演示。这个演示必须稳定完成下面五个动作：

1. 用户初始化钱包，并设置一次意图。
2. 用户关闭前端或断开网络。
3. 源链触发一个外部事件。
4. Reactive 监听器收到事件后自动驱动钱包执行预设逻辑。
5. 用户重新打开前端，看到执行记录、余额变化、策略执行次数都已更新。

这一版的核心价值不是“钱包功能丰富”，而是“钱包在用户离线后继续执行用户意图”。

2. MVP 范围定义

必须做

- 一个可持有资产的钱包合约
- 一个最小策略：固定金额转账或 DCA
- 一个源链事件触发器
- 一个 Reactive 监听合约
- 一个最小前端：创建钱包、设置意图、查看执行结果

不要做

- 多策略并行系统
- 复杂收益图表
- 完整多链资产管理
- 社交恢复、多签、AA 全套能力
- 复杂价格源接入

3. 推荐技术框架

层	推荐技术	原因
智能合约	Solidity + Foundry + OpenZeppelin + reactive-lib	你们已经有 Foundry 和 Reactive 合约经验，继续用同一套栈效率最高。
前端	Vite + React + TypeScript	比 Next.js 更轻，适合 7 到 10 天做 MVP，全部页面都可以用客户端模式处理。

层	推荐技术	原因
链交互	viem + wagmi	现代以太坊前端常用方案，读写合约、监听交易、接钱包都够用。
样式	Tailwind CSS	开发快，做演示页和状态卡片效率高。
状态管理	Zustand	轻量，适合处理“当前钱包 / 当前策略 / 当前网络状态”。
后端	不做独立后端	MVP 直接从链上读状态，不引入额外复杂度。
数据源	合约事件 + 直接读取状态	演示阶段不必引入 indexer，减少故障点。

实现优先级：先把链上闭环做通，再做前端；先做固定转账，再考虑 DCA 包装；先做单钱包实例，再考虑工厂模式。

4. 系统架构

建议这一版只保留三个核心合约，控制复杂度：

合约	网络	职责
<code>WillLeadSignalEmitter</code>	Origin Chain	发出策略触发事件，例如 <code>StrategySignal(wallet, amount, nonce)</code> 。
<code>WillLeadReactiveListener</code>	Reactive Lasna	订阅源链事件，在 <code>react(LogRecord)</code> 中解析 payload，并发出 <code>Callback(...)</code> 。
<code>WillLeadWallet</code>	Destination Chain	持有资金、保存策略配置、接收 callback 并执行固定金额转账。

推荐链路

- Origin: Base Sepolia
- Reactive: Reactive Lasna
- Destination: Ethereum Sepolia

这样可以复用你们已经跑通过的链路和 callback proxy 配置，少踩基础设施坑。

5. 合约设计

5.1 WillLeadSignalEmitter

这是演示专用信号源，不需要接真实价格或复杂 oracle。它的任务就是稳定地产生可监听事件。

```
event StrategySignal(
    address indexed wallet,
    address indexed token,
    address indexed recipient,
    uint256 amount,
    uint256 executionNonce,
    uint256 emittedAt
);
```

```
function emitSignal(
    address wallet,
    address token,
    address recipient,
    uint256 amount,
    uint256 executionNonce
) external;
```

演示时，外部账户调用 `emitSignal(...)` 就等于“市场条件达成”或“策略 tick 到达”。

5.2 WillLeadWallet

这个钱包不追求完整钱包能力，只做这几个动作：

- 接受原生资产或测试代币
- 保存一条策略配置
- 接收 Reactive callback
- 执行一次固定金额转账
- 记录最近一次执行结果

建议保存的核心状态：

```
struct StrategyConfig {
    bool enabled;
    address token;
    address recipient;
    uint256 amountPerExecution;
    uint256 maxExecutions;
    uint256 executedCount;
}

address public owner;
address public callbackProxy;
address public authorizedRvmId;
StrategyConfig public strategy;
uint256 public lastExecutionNonce;
uint256 public lastExecutedAt;
bytes32 public lastSignalHash;
```

建议暴露的核心函数：

- `configureStrategy(...)`
- `pauseStrategy()`
- `deposit()` 或直接 `receive()`
- `callback(address,uint256,address,address,uint256,uint256)`
- `withdraw(...)` 仅 owner 可调用

`callback(...)` 内部需要做这些检查：

- 仅 callback proxy 可调用
- 仅授权的 `rvmId` 可通过
- 策略必须启用
- `executedCount < maxExecutions`
- 防重复执行：`executionNonce` 或 `signalHash` 不能重复

5.3 WillLeadReactiveListener

这个合约负责把源链事件转换成目标链钱包可执行的 callback payload。

```
function react(LogRecord calldata logRecord) external vmOnly {
    // 1. 校验来源链和来源合约
    // 2. 解析 wallet / token / recipient / amount / executionNonce
    // 3. abi.encodeWithSignature("callback(...)")
    // 4. emit Callback(destinationChainId, wallet, gasLimit, payload)
}
```

建议 callback gas limit 直接定成 `1_000_000`，不要压太低，减少 callback 被忽略的风险。

6. 前端设计

前端只做 3 个页面或 3 个主视图，不要发散。

页面	必须展示的内容
Wallet Setup	连接钱包、创建或绑定 WillLeadWallet、展示钱包地址和余额。
Intent Setup	配置策略：收款地址、每次金额、最大执行次数、启用状态。
Execution Dashboard	策略状态、执行次数、最后一次执行时间、最近一次交易哈希、钱包余额。

前端侧推荐的目录结构：

```
src/
  app/
    App.tsx
  components/
    WalletHeader.tsx
    StrategyForm.tsx
    ExecutionPanel.tsx
    ProofPanel.tsx
  contracts/
    abi/
      addresses.ts
  hooks/
    useWalletState.ts
    useStrategyState.ts
    useExecutionHistory.ts
  lib/
    wagmi.ts
    viem.ts
  store/
    walletStore.ts
```

7. 链上执行流程

```
用户设置策略
-> WillLeadWallet 保存 StrategyConfig
-> 用户关闭前端
-> 外部账户调用 WillLeadSignalEmitter.emitSignal(...)
-> 源链发出 StrategySignal
-> WillLeadReactiveListener 监听该事件
-> react(LogRecord) 解析参数
-> emit Callback(...)
-> 目标链 callback proxy 调用 WillLeadWallet.callback(...)
-> 钱包执行固定金额转账
-> 钱包状态更新 executedCount / lastExecutedAt / lastSignalHash
-> 用户重新打开前端后读取最新链上状态
```

8. 关键实现细节

8.1 callback 资金路径

目标链 callback 合约不能只靠“直接转账到钱包地址”来准备 callback 资金。脚本里应该使用官方 callback proxy 的 `depositTo(walletAddress)`，确保 callback proxy 能够在目标链为该钱包执行回调。

8.2 为什么不用真实价格源

MVP 的目标是证明“离线后仍执行”，不是证明你们已经做出完整量化系统。接真实价格源会带来额外不稳定因素，不利于黑客松演示。第一版应该优先让信号源可控。

8.3 防止重复执行

钱包内必须保存去重状态，例如 `lastExecutionNonce` 或 `processedSignals[signalHash]`。Reactive 体系是事件驱动，不应假设跨链链路天然具备原子回滚语义。

8.4 用户离线的正确说法

演示时要明确：离线的是用户前端，不是整个系统。真正成立的命题是“用户端不需要持续在线，Reactive 网络仍可继续驱动链上执行”。

9. 推荐开发顺序

阶段	目标
Day 1	定稿合约接口，写出 <code>SignalEmitter</code> 、 <code>WillLeadWallet</code> 、 <code>ReactiveListener</code> 基础骨架。
Day 2	跑通最小闭环：源链事件 -> Reactive -> callback -> 目标链状态变化。
Day 3	让钱包 callback 后真的执行固定金额转账，并记录执行状态。
Day 4	补充资金路径： <code>depositTo(wallet)</code> 、 <code>pause</code> 、 <code>withdraw</code> 、去重逻辑。
Day 5	前端完成 Wallet Setup + Intent Setup + Execution Dashboard。
Day 6	串完整 demo：先配置、再离线、再触发事件、再回来读取结果。
Day 7	只做演示稳定性：准备浏览器链接、交易哈希、录屏流程、故障兜底。
Day 8-10	优化 UI、补充项目文案、准备答辩材料和问题清单。

10. 演示脚本建议

1. 打开前端，创建或连接 WillLead 钱包。
2. 设置一个固定金额策略，比如“收到 `StrategySignal` 后转出 0.01 测试 ETH”。
3. 展示当前策略状态为 `enabled`，执行次数为 `0`。
4. 关闭网页，或者断开本机网络。
5. 在另一窗口或命令行里触发源链事件。
6. 展示区块浏览器和 Reactscan：源链事件、Reactive callback、目标链交易都真实发生。
7. 重新打开前端，读取最新链上状态。
8. 展示执行次数已增加、余额已变化、最后一次交易哈希已记录。

11. 关键风险与应对

风险	应对
callback 不落地	优先检查目标链 callback proxy 资金是否通过 <code>depositTo(...)</code> 准备完成。
react() 直接 revert	检查订阅来源、目标钱包映射、callback payload 编码和 chain id 配置。
策略被重复执行	加入 nonce 或 signal hash 去重。
评委误解为后端 bot	演示时必须真正关闭前端，并用链上证据证明执行仍在发生。
开发范围失控	只保留单钱包、单策略、单信号源。

12. 最终交付标准

满足以下条件，就可以认为 WillLead MVP 已经可演示：

- 用户能在前端设置一次策略。
- 前端关闭后，外部事件仍能驱动链上执行。
- 目标链钱包会发生真实状态变化和真实交易。
- 用户重新上线后可以看到完整执行结果。
- 全流程有浏览器链接和交易哈希作为证明。

一句话定义这个 MVP： WillLead is a reactive-native autonomous wallet prototype that keeps executing user intent onchain even after the frontend goes offline.

文档生成时间：2026-03-10。这个版本用于黑客松 MVP 开发，不追求完整钱包能力，只追求稳定演示和高效落地。